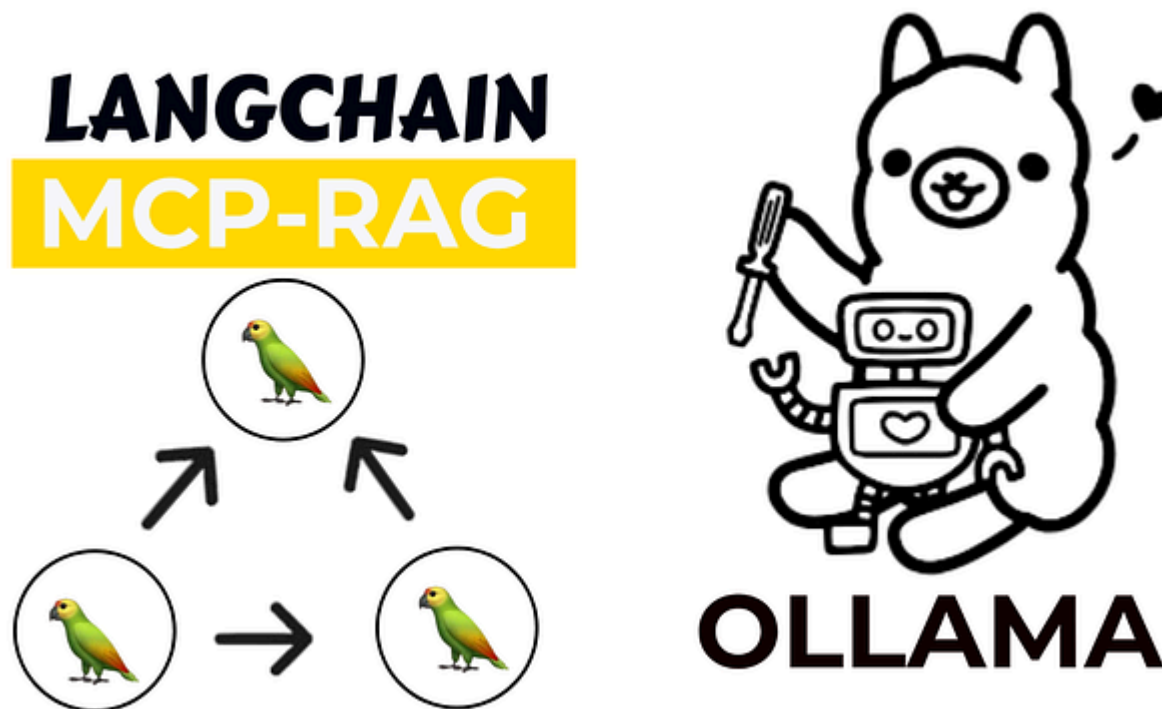


[< Go to the original](#)

## LangChain + MCP + RAG + Ollama = The Key To Powerful Agentic AI

In this video, I have a super quick tutorial showing you how to create a multi-agent chatbot using LangChain, MCP, RAG, and Ollama to build...



Follow



Data Science Collective a11y-light ~12 min read · April 19, 2025 (Updated: April 19, 2025) · Free: No

In this video, I have a super quick tutorial showing you how to create a multi-agent chatbot using LangChain, MCP, RAG, and Ollama to build a powerful agent chatbot for your business or personal use.

If this is your first time watching me, I highly recommend checking out my previous video. I made a video about MCP, which became a big hit in the AI community.

MCP, RAG, and Agent have been very popular recently. Almost everyone working on AI is talking about this topic. At the same time, a large number of MCP tools have been born almost every day.

This is because RAG relies on search tools to acquire knowledge, and MCP can standardise calling these tools.

The tool chain coordinated by MCP requires real-time data support, and RAG can dynamically provide knowledge context. For example, the Agent calls the Web

Similarly, in supply chain optimisation, MCP calls the inventory API to obtain real-time inventory data

Not long ago, Mistral came up with another "small and beautiful" product: Mistral Small 3.1 was released and is open source:

### mistral-small3.1

Building upon Mistral Small 3, Mistral Small 3.1 (2503) adds state-of-the-art vision understanding and enhances long context capabilities up to 128k tokens without compromising text performance.

[vision](#)[tools](#)[24b](#)[↓ 20.6K Pulls](#)[🔖 5 Tags](#)[🕒 Updated 5 days ago](#)

Mistral Small 3.1 is an upgraded version of Mistral Small 3, with 24B (24 billion) parameters, designed to provide efficient, low-latency and high-performance generative AI solutions. The model is released under the Apache 2.0 license, allowing for commercial purposes.

---

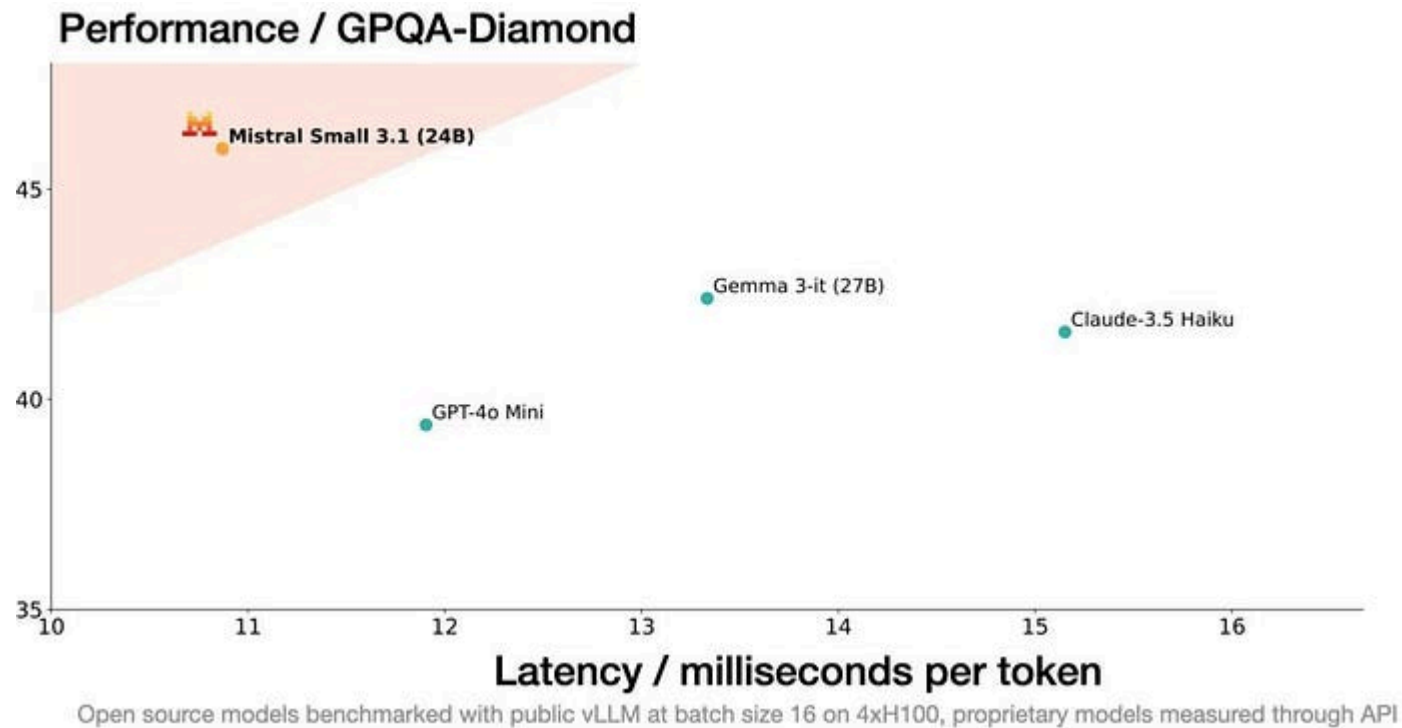
graphics card or a Mac with 32GB RAM, which is 3 times faster than Llama 3.3 70B with the same number of parameters on the same hardware.

**Freedium**

ko-fi librepay patreon



dominates the upper left corner, with speed and knowledge double buffs.



So, let me give you a quick demo of a live chatbot to show you what I mean.



I will ask the chatbot a question: "Get the latest news about LLMs?" Feel free to ask any questions you want.

If you look at how the chatbot generates the output, you will see that when a user enters a query, the Agent searches the web, extracts URLs, fetches the full

These chunks are embedded using Mistral AI or ollama embedding. Feel free to use any embedding you want, and store it in a FAISS vector database for fast semantic retrieval.

One of the problems I faced was that when we wrapped our search and RAG functions as MCP tools, we ran into errors like "Attempted to exit cancel scope in a different task than it was entered in" and "ClosedResourceError." MCP couldn't properly handle the async connections needed for web searches.

The way I solved this was by modifying the code to create fresh connections for each tool call instead of maintaining long-lived connections. This isolated each operation in its context.

Then it performs a RAG-based search to return the most relevant information from the fetched content, alongside traditional search results. All core functions — like `search_web`, `get_web_content`, `create_rag`, and `search_rag` — are wrapped with error handling and can run independently or through a Fastmcp-powered server with real-time interaction via Server-Sent Events (SSE).



---

RAG and OpenSource to create a powerful Agentic chatbot.

## MCP Vs RAG

The differences between MCP and RAG reflect their different positioning in practical applications. MCP is particularly suitable for scenarios that require LLM to perform complex operations, such as when developing AI agents, where the agent may need to call external tools to complete tasks, such as extracting data from a database or using a constraint solver to solve a problem.

On the other hand, RAG is more suitable for scenarios where information needs to be kept up to date and accurate, such as when an enterprise chatbot needs to answer questions related to products or services, which may be beyond the scope of LLM's training data.

## How to combine them

An interesting observation is that the two can be used in combination. For example, an AI agent can use MCP to call a retrieval tool (such as a web search)

This combination may become more and more common in AI applications in 2025, especially in scenarios that require dynamic interaction and real-time information updates.

MCP's success relies on industry support for the protocol and addressing security and scalability issues, while RAG's popularity benefits from its cost-effectiveness and ease of implementation.

However, the versatility of MCP may make it a more widely integrated standard in the future, while RAG is likely to continue to dominate in knowledge-intensive tasks.

## Let's start coding

Let us now explore step by step and unravel the answer to how to create the MCP APP. We will install the libraries that support the model. For this, we will do a pip install requirements

Copy

The next step is the usual one, where we will import the relevant libraries, the significance of which will become evident as we proceed.

We initiate the code by importing classes from

FireCrawlLoader is a powerful tool that allows you to crawl and convert any website into Markdown format. It efficiently handles all accessible subpages, providing clean markdown and metadata for each page.

**exa\_py** allows users to search using complete sentences and natural language, and can also simulate the way people share and talk about links on the Internet to query content.

MCP is an open protocol that standardizes how applications provide context to LLMs.

Copy

```
#mcp_server.py
import asyncio
from mcp.server.fastmcp import FastMCP
import rag
```

```
import os
```

```
#rag.py
```

```
from langchain_ollama import OllamaEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
import search
from langchain_core.documents import Document
import os
import asyncio
```

```
#search.py
```

```
import asyncio
from dotenv import load_dotenv
import os
from exa_py import Exa
from typing import List, Tuple
from langchain_core.documents import Document
from langchain_community.document_loaders.firecrawl import FireCrawlLoader
import requests
```

```
#agent.py
```

```
import asyncio
import os
import sys
```

---

Import Rag

## Server.py

I created a Fastmcp-powered web search tool that integrates real-time web capabilities using the Exa and Firecrawl APIs, enhanced with RAG (Retrieval-Augmented Generation) for more relevant results.

The `FastMCP` for instance, named `web_search`, supports advanced filtering (by domain, keywords, and dates) and delivers structured outputs including titles, URLs, and summaries.

I designed a `search_web_tool` function, registered with `@mcp.tool()`, which accepts a query, performs the search, extracts and validates URLs, and runs RAG processing using `create_rag` and `search_rag` to retrieve semantically relevant content.

I included robust error handling for no results, invalid URLs, and search failures. I also developed `get_web_content_tool` to fetch and return full webpage content with a 15-second timeout, handling various errors gracefully.

server using SSE (Server-Sent Events) for real-time tool interaction.

Copy

```
mcp = FastMCP(
    name="web_search",
    version="1.0.0",
    description="Web search capability using Exa API , Firecrawl API that prov
)

@mcp.tool()
async def search_web_tool(query: str) -> str:
    logger.info(f"Searching web for query: {query}")
    formatted_results, raw_results = await search.search_web(query)

    if not raw_results:
        return "No search results found."

    urls = [result.url for result in raw_results if hasattr(result, 'url')]
    if not urls:
        return "No valid URLs found in search results."

    vectorstore = await rag.create_rag(urls)
    rag_results = await rag.search_rag(query, vectorstore)

    # You can optionally include the formatted search results in the output
    full_results = f"{formatted_results}\n\n### RAG Results:\n\n"
    full_results += '\n---\n'.join(doc.page_content for doc in rag_results)
```

```
@mcp.tool()
async def get_web_content_tool(url: str) -> str:
    try:
        documents = await asyncio.wait_for(search.get_web_content(url), timeout)
        if documents:
            return '\n\n'.join([doc.page_content for doc in documents])
        return "Unable to retrieve web content."
    except asyncio.TimeoutError:
        return "Timeout occurred while fetching web content. Please try again."
    except Exception as e:
        return f"An error occurred while fetching web content: {str(e)}"
```

## Rag.py

So, I designed a RAG pipeline that creates a FAISS vector store from a list of URLs using Mistral embedding or Ollama embedding and async content retrieval. Then I create `create_rag`. The function begins by initializing an embedding model, defaulting to your desired embedding, with support for custom API keys and endpoints, and processes input in token batches of 64.

I then fetch web content asynchronously from all URLs to improve speed, aggregating the retrieved `Document` objects. These documents are split into

with LLM context limits.

The split chunks are converted into embeddings and stored in a FAISS index for fast similarity search. I added robust error handling to catch issues during embedding, fetching, or storage.

I also created `create_rag_from_documents`, a more flexible alternative that accepts already-fetched documents, ideal for offline or cached workflows. Finally, I built the `search_rag` function to semantically retrieve the top 3 most relevant chunks from FAISS based on cosine similarity, enabling efficient and meaningful document-level question answering.

Copy

```
async def create_rag(links: list[str]) -> FAISS:
    try:
        model_name = os.getenv("MODEL", "text-embedding-ada-002")
        # Change any embedding you want, whether Ollama or MistralAIEmbeddings
        # embeddings = MistralAIEmbeddings(
        #     model="mistral-embed",
        #     chunk_size=64
        # )
        embeddings = OpenAIEmbeddings(
            model=model_name,
```



```

        chunk_size=1000,
    )
    documents = []
    # Use asyncio.gather to process all URL requests in parallel
    tasks = [search.get_web_content(url) for url in links]
    results = await asyncio.gather(*tasks)
    for result in results:
        documents.extend(result)

    # Text chunking processing
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=10000,
        chunk_overlap=500,
        length_function=len,
        is_separator_regex=False,
    )
    split_documents = text_splitter.split_documents(documents)
    # print(documents)
    vectorstore = FAISS.from_documents(documents=split_documents, embedding=embedding)
    return vectorstore
except Exception as e:
    print(f"Error in create_rag: {str(e)}")
    raise

async def create_rag_from_documents(documents: list[Document]) -> FAISS:
    """
    Create a RAG system directly from a list of documents to avoid repeated web requests.

    Args:
    """

```

```
#####  
FAISS: Vector store object  
#####  
try:  
    model_name = os.getenv("MODEL")  
    embeddings = OpenAIEmbeddings(  
        model=model_name,  
        openai_api_key=os.getenv("OPENAI_API_KEY"),  
        openai_api_base=os.getenv("OPENAI_API_BASE"),  
        chunk_size=64  
    )  
  
    # Text chunking processing  
    text_splitter = RecursiveCharacterTextSplitter(  
        chunk_size=10000,  
        chunk_overlap=500,  
        length_function=len,  
        is_separator_regex=False,  
    )  
    split_documents = text_splitter.split_documents(documents)  
  
    vectorstore = FAISS.from_documents(documents=split_documents, embedding=embeddings)  
    return vectorstore  
except Exception as e:  
    print(f"Error in create_rag_from_documents: {str(e)}")  
    raise
```

## Search.py

I developed a web search and content retrieval system using the Exa and FireCrawl APIs, designed to perform intelligent web queries, extract relevant information, and format it for use in downstream tasks like Retrieval-Augmented Generation (RAG).

Then I created a configuration dictionary to manage search defaults and constants such as maximum retry attempts and request timeouts.

The core of the system includes three main functions: `search_web()`, which sends a query to the Exa API and returns both formatted Markdown summaries and raw results; `format_search_results()`, which structures the results with titles, URLs, publication dates, and summaries in a clean Markdown format; and `get_web_content()`, which uses FireCrawl to fetch full-page content from URLs and convert it into structured `Document` objects.

I implemented robust error handling and retry logic to manage unsupported websites and transient failures. The whole pipeline begins with a user-provided query, retrieves summarized results from Exa, and scrapes their content with

Copy

```
# Load .env variables
load_dotenv(override=True)

# Initialize the Exa client
exa_api_key = os.getenv("EXA_API_KEY", " ")
exa = Exa(api_key=exa_api_key)
os.environ['FIRECRAWL_API_KEY'] = ''

# Default search config
websearch_config = {
    "parameters": {
        "default_num_results": 5,
        "include_domains": []
    }
}

# Constants for web content fetching
MAX_RETRIES = 3
FIRECRAWL_TIMEOUT = 30 # seconds

async def search_web(query: str, num_results: int = None) -> Tuple[str, list]:
    """Search the web using Exa API and return both formatted results and raw results"""
    try:
        search_args = {
            "num_results": num_results or websearch_config["parameters"]["default_num_results"]
        }
```

```

search_results = exa.search_and_contents(
    query,
    summary={"query": "Main points and key takeaways"},
    **search_args
)

formatted_results = format_search_results(search_results)
return formatted_results, search_results.results
except Exception as e:
    return f"An error occurred while searching with Exa: {e}", []

def format_search_results(search_results):
    if not search_results.results:
        return "No results found."

    markdown_results = "### Search Results:\n\n"
    for idx, result in enumerate(search_results.results, 1):
        title = result.title if hasattr(result, 'title') and result.title else ""
        url = result.url
        published_date = f" (Published: {result.published_date})" if hasattr(result, 'published_date') else ""

        markdown_results += f"**{idx}.** [{title}]({url}){published_date}\n"

        if hasattr(result, 'summary') and result.summary:
            markdown_results += f"> **Summary:** {result.summary}\n\n"
        else:
            markdown_results += "\n"

    return markdown_results

```

```
GET WEB CONTENT AND CONVERT TO DOCUMENT LIST.  
for attempt in range(MAX_RETRIES):  
    try:  
        # Create FireCrawlLoader instance  
        loader = FireCrawlLoader(  
            url=url,  
            mode="scrape"  
        )  
  
        # Use timeout protection  
        documents = await asyncio.wait_for(loader.aload(), timeout=FIRECRAWL_TIMEOUT)  
  
        # Return results if documents retrieved successfully  
        if documents and len(documents) > 0:  
            return documents  
  
        # Retry if no documents but no exception  
        print(f"No documents retrieved from {url} (attempt {attempt + 1}/{MAX_RETRIES})")  
        if attempt < MAX_RETRIES - 1:  
            await asyncio.sleep(1) # Wait 1 second before retrying  
            continue  
  
    except requests.exceptions.HTTPError as e:  
        if "Website Not Supported" in str(e):  
            # Create a minimal document with error info  
            print(f"Website not supported by FireCrawl: {url}")  
            content = f"Content from {url} could not be retrieved: Website not supported"  
            return [Document(page_content=content, metadata={"source": url})]  
        else:
```

```
if attempt < MAX_RETRIES - 1:
    await asyncio.sleep(1)
    continue
raise
except Exception as e:
    print(f"Error retrieving content from {url}: {str(e)} (attempt {att
    if attempt < MAX_RETRIES - 1:
        await asyncio.sleep(1)
        continue
    raise

# Return empty list if all retries failed
return []
```

## Agent.py

**Note: You can still run Agent.py successfully without calling MCP, but you will not be able to fetch data unless the server is running**

Then I created a system that first checks for a search query either from command line arguments or user input, then initiates a web search through the search module.

system then performs a semantic search on this vector store to find the most relevant information related to the original query.

I made sure to include proper error handling and user feedback, displaying both traditional search results and AI-enhanced RAG results separately.

The entire process runs asynchronously using asyncio for efficient I/O operations, and I structured the output to clearly distinguish between different types of results for a better user experience.

Copy

```
import asyncio
import os
import sys

# Import search and RAG modules directly
import search
import rag

# Configure environment

async def main():
```



```
query = " ".join(sys.argv[1:])
else:
    query = input("Enter search query: ")

print(f"Searching for: {query}")

try:
    # Call search directly
    formatted_results, raw_results = await search.search_web(query)

    if not raw_results:
        print("No search results found.")
        return

    print(f"Found {len(raw_results)} search results")

    # Extract URLs
    urls = [result.url for result in raw_results if hasattr(result, 'url')]
    if not urls:
        print("No valid URLs found in search results.")
        return

    print(f"Processing {len(urls)} URLs")

    # Create RAG
    vectorstore = await rag.create_rag(urls)
    rag_results = await rag.search_rag(query, vectorstore)

    # Format results
```

```
print("\n=== RAG Results ===")
for doc in rag_results:
    print(f"\n---\n{doc.page_content}")

except Exception as e:
    print(f"Error: {e}")
    import traceback
    traceback.print_exc()

if __name__ == "__main__":
    asyncio.run(main())
```

## Conclusion :

So far, you can see that combining the RAG with the MCP server significantly improves the performance of the AI by providing the agent with the required knowledge (through retrieval) and contextual awareness of the situation (through memory and data integration).

The AI-driven system becomes more autonomous and useful. It can act as a researcher, assistant, or analyst, not only having access to information at all times, but also understanding when and how to apply it.

## Reference :

- <https://modelcontextprotocol.io/introduction>
- <https://github.com/rishikavikondala/mcp-server-aws>
- <https://docs.cursor.com/context/model-context-protocol>

 *I am an AI Generative expert! If you want to collaborate on a project, drop an [inquiry here](#) or Book a [1-on-1 Consulting Call With Me](#).*

 *Feel free to check out my other articles:*

### **Pydantic AI + MCP + Advanced Web Scraping = The Key To Powerful Agentic AI**

In this video, I have a super quick tutorial showing you how to create a multi-agent chatbot using Pydantic AI, MCP...

medium.com

### **LangGraph + MCP + Ollama: The Key To Powerful Agentic AI**

medium.com

**Gemma 3 + MistralOCR + RAG Just Revolutionized Agent OCR  
Forever**

Not a Month Ago, I made a video about Ollama-OCR. Many of you like  
this video

towardsai.net

[#data-science](#)   [#artificial-intelligence](#)   [#machine-learning](#)   [#technology](#)   [#programming](#)